

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

**Лабораторная работа №2
«Реализация микросервисов»**

Выполнила:

Гайдук Алина Сергеевна

Группа: К3341

Проверил:

Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

В рамках лабораторной работы №2 я реализовала разделение backend-приложения из ЛР1 на набор микросервисов. В качестве архитектурной основы я использовала технический дизайн из ДЗ4: пять предметных сервисов, API Gateway, принцип database-per-service и синхронное межсервисное взаимодействие по HTTP.

Цель работы состояла в том, чтобы каждый сервис получил собственную зону ответственности, собственную модель данных и минимальный набор внутренних контрактов для взаимодействия с остальной системой.

При реализации я придерживалась следующих требований:

1. сохранить публичный контракт API под префиксом /api/v1;
2. разделить данные между сервисами без внешних ключей между разными базами;
3. оставить API Gateway в роли BFF/reverse-proxy, не перенося в него бизнес-логику;
4. защитить внутренние HTTP-эндпоинты заголовком X-Service-Token;
5. использовать слоистую структуру domain, usecase, infrastructure и transport/http;
6. проверить изменения unit-тестами, сборкой сервисов и линтером.

Ход работы

Архитектурные принципы

Сначала я перенесла монолитное приложение из ЛР1 в lab2 и выделила границы сервисов по данным и бизнес-возможностям. Я исходила из того, что сервис должен владеть теми данными, которые он изменяет чаще всего и для которых он способен самостоятельно проверять инварианты.

В итоговой реализации публичный вход в систему находится в gateway. Он маршрутизирует запросы на сервис-владелец, а не собирает бизнес-ответы самостоятельно. Внутренние запросы между сервисами идут по /internal/v1 и используются для проверок существования, получения кратких представлений сущностей, пакетного обогащения авторов и получения счетчиков активности.

Для кода я оставила чистую слоистую структуру. Доменные модели лежат в internal/domain, сценарии приложения в internal/usecase, работа с БД и внешними сервисами в internal/infrastructure, а HTTP-доставка и хендлеры в internal/transport/http. Ранее существовавшие пакеты internal/api и internal/interfaces были убраны, чтобы HTTP-слой имел один источник истины.

Состав микросервисов

В результате я получила шесть запускаемых приложений: gateway и пять микросервисов. Для каждого сервиса создан отдельный endpoint в cmd/<service>, отдельный usecase-слой, репозиторий и HTTP router/handler.

Таблица 1 – зоны ответственности сервисов

Сервис	Ответственность	Данные сервиса
gateway	Единая публичная точка входа /api/v1; reverse-proxy на сервисы-владельцы.	Своей БД нет.
identity-service	Регистрация, вход, refresh JWT, профили пользователей, подписки.	users, refresh_tokens, follows.
catalog-service	Справочники рецептов и проверка reference id.	dish_types, difficulties, tags, ingredients, measurement_units.
recipe-service	Рецепты, шаги, состав, теги рецептов, выдача рецептов автора.	recipes, recipe_steps, recipe_ingredients, recipe_tags.
blog-service	Посты блога и выдача постов автора.	posts.
engagement-service	Комментарии, лайки рецептов и постов, сохраненные рецепты, счетчики активности.	comments, recipe_likes, post_likes, saved_recipes.

Разделение данных

Я реализовала database-per-service в docker-compose через один контейнер PostgreSQL и пять отдельных баз данных. Такой вариант удобен для лабораторной работы: инфраструктурно он проще, но логически сохраняет независимость данных сервисов. В compose создаются базы recipehub_identity, recipehub_catalog, recipehub_recipe, recipehub_blog и recipehub_engagement.

Каждый сервис получает свой *_SERVICE_DATABASE_URL и открывает только свою базу. Между базами нет внешних ключей: например, recipe-service хранит author_id как логический идентификатор пользователя и проверяет его через identity-service. Engagement-service аналогично хранит target_id для

рецепта или поста и проверяет существование цели через recipe-service или blog-service.

Миграции также разнесены по репозиториям: identityrepo, catalogrepo, reciperepo, blogrepo и engagementrepo. Это делает владение схемой явным: сервис не должен менять таблицы другого сервиса.

Публичный API и gateway

Публичный контракт сохранен под /api/v1. Gateway не содержит usecase-слоя и не подключается к базе данных. Его задача — принять публичный запрос и отправить его в сервис-владелец. Так я сохранила совместимость внешнего API и одновременно не вернула монолитную бизнес-логику в точку входа.

Таблица 2 — маршрутизация публичного API через gateway

Группа маршрутов /api/v1	Сервис-владелец
/auth/*, /users/me, /users/{id}, /users/{id}/followers, /users/{id}/following, /users/{id}/follow	identity-service
/dish-types, /difficulties, /tags, /ingredients	catalog-service
/recipes, /recipes/{id}, /users/{id}/recipes	recipe-service
/posts, /posts/{id}, /users/{id}/posts	blog-service
/recipes/{id}/comments, /posts/{id}/comments, /comments/{id}, лайки и сохраненные рецепты	engagement-service

Внутренние API и взаимодействие сервисов

Для межсервисных вызовов я добавила HTTP-клиенты в internal/infrastructure/clients. Все они добавляют заголовок X-Service-Token, а на стороне принимающих сервисов internal routes закрыты service-token middleware. Это отделяет публичную авторизацию пользователя по JWT от доверенных сервисных вызовов между сервисами.

Таблица 3 — основные внутренние взаимодействия

Кто вызывает	Кого вызывает	Зачем
recipe-service	identity-service	Проверить существование автора и получить краткие данные авторов для списков рецептов.

recipe-service	catalog-service	Проверить dish_type_id, difficulty_id, tag_ids, ingredient_ids и unit_ids при создании или изменении рецепта.
recipe-service	engagement-service	Получить likes_count, comments_count, is_liked и is_saved для рецептов.
blog-service	identity-service	Проверить автора и обогатить посты краткими данными пользователя.
blog-service	engagement-service	Получить likes_count, comments_count и is_liked для постов.
engagement-service	recipe-service / blog-service	Проверить существование цели комментария или лайка и получить краткий рецепт для списка сохраненного.
identity-service	recipe-service	Получить recipes_count для публичного профиля пользователя.

Для внутренних контрактов реализованы ручки /internal/v1/users/{id}/exists, /internal/v1/users/batch, /internal/v1/catalog/validate-ids, /internal/v1/recipes/{id}/exists, /internal/v1/recipes/{id}/brief, /internal/v1/authors/{userId}/recipe-count, /internal/v1/posts/{id}/exists, /internal/v1/posts/{id}/brief, /internal/v1/stats/recipes/batch и /internal/v1/stats/posts/batch.

Реализация слоев

В каждом сервисе я придерживалась одинаковой схемы. Domain содержит типы предметной области без HTTP и GORM-зависимостей. Usecase реализует сценарии и принимает интерфейсы репозиторий и клиентов. Infrastructure содержит GORM-репозитории, миграции и HTTP-клиентов к другим сервисам. Transport/http содержит router, handler и DTO конкретного HTTP API.

Например, recipe-service при создании рецепта сначала проверяет автора через identity-service, затем валидирует справочники через catalog-service, после чего сохраняет собственные данные в своей базе. При выдаче списка рецептов он дополнительно обращается в engagement-service за счетчиками и пользовательскими флагами. Такой поток показывает, что сервис владеет рецептами, но не пытается владеть пользователями, справочниками или лайками.

Для identity-service я добавила агрегацию recipes_count через recipe-service. Для blog-service и recipe-service добавлены клиенты engagement-service, чтобы

поля `likes_count`, `comments_count`, `is_liked` и `is_saved` больше не были заглушками. В `engagement-service` я обработала повторные `like/save` как доменную ошибку `ErrAlreadyExists`, которая на HTTP-уровне превращается в `409 Conflict`, а не в сырой `DB error`.

Инфраструктура запуска

Для локального запуска всех микросервисов я добавила `docker-compose`. В нем описаны `PostgreSQL`, `gateway`, `identity-service`, `catalog-service`, `recipe-service`, `blog-service` и `engagement-service`. У каждого сервиса свой порт на хосте, но внутри `compose` все приложения слушают `:8080`. Сервисные URL передаются через `IDENTITY_SERVICE_URL`, `CATALOG_SERVICE_URL`, `RECIPE_SERVICE_URL`, `BLOG_SERVICE_URL` и `ENGAGEMENT_SERVICE_URL`.

Общая конфигурация вынесена в `internal/config`. Функция `LoadService` читает `service-specific` переменные вида `IDENTITY_SERVICE_DATABASE_URL` и общие параметры `JWT` и `SERVICE_TOKEN`. Общий запуск HTTP-сервера находится в `internal/platform/server`, а открытие `PostgreSQL` — в `internal/platform/postgres`. Благодаря этому `entrypoint` каждого сервиса остается коротким и состоит только из сборки зависимостей.

Очистка legacy-кода

После стабилизации микросервисного пути я удалила `legacy`-части монолита из `lab2`: старый `cmd/service`, `internal/httpserver`, старые `handler/deps/mapper` в `internal/api` и общий `database-store` из `internal/infrastructure/database`. Оставшиеся HTTP-хендлеры находятся в `internal/transport/http/<service>`, поэтому сейчас в проекте нет двух конкурирующих HTTP-слоев.

Такой шаг важен для сдачи ЛР2: структура проекта теперь показывает именно микросервисную реализацию, а не монолит с добавленными рядом сервисами.

Проверка корректности

Я проверила работу на уровне `unit` и `router`-тестов. Интеграционные `smoke`-тесты намеренно не добавляла, так как для этой итерации достаточно `unit`-покрытия и проверки маршрутизации. Тесты покрывают `usecase`-логику `catalog`, `identity` и `engagement`, а также доступность публичных и внутренних маршрутов у `gateway`, `identity`, `catalog`, `recipe`, `blog` и `engagement`.

Перед завершением я выполнила стандартный набор проверок:

```
gofmt для Go-файлов;
```

```
go test ./...;
```

```
go build ./cmd/...;
```

```
golangci-lint run ./...;
```

```
docker compose config --quiet;
```

поиск старых импортов `internal/api` и `internal/interfaces/http` через `rg`.

Все проверки прошли успешно. После финального переноса HTTP-слоя старые импорты `internal/api` и `internal/interfaces/http` в `lab2` не находятся, а сервисы собираются через новые `entrypoint`'ы `cmd/gateway`, `cmd/identity-service`, `cmd/catalog-service`, `cmd/recipe-service`, `cmd/blog-service` и `cmd/engagement-service`.

Вывод

В ходе лабораторной работы я реализовала микросервисную версию backend-приложения `RecipeHub` в соответствии с техническим дизайном из ДЗ4. Я выделила `identity-service`, `catalog-service`, `recipe-service`, `blog-service`, `engagement-service` и `gateway`, разделила данные по принципу `database-per-service` и настроила внутренние HTTP-контракты между сервисами.

Главный результат работы — приложение перестало быть монолитом не только по структуре папок, но и по владению данными. Рецепты больше не читают напрямую пользователей или лайки из общей БД, `engagement-service` не хранит контент рецепта или поста, а `gateway` не принимает бизнес-решения. Сервисы взаимодействуют через явные `internal API`, защищенные `X-Service-Token`.

Также я привела структуру кода к единому виду: хендлеры и маршрутизаторы находятся в `internal/transport/http`, доменная логика отделена от доставки, а `legasy`-код ЛР1 удален из `lab2`. После изменений проект проходит тесты, сборку всех сервисов, линтер и проверку `docker-compose`, поэтому текущая реализация соответствует требованиям ЛР2 и выбранной микросервисной архитектуре.